Corporate Evasion Decoder: Detecting Evasive Communication in Earnings Call Q&A Sessions

Wei-Lin Wen (Student ID: 314551086)
National Yang Ming Chiao Tung University

Dataset Repository: https://github.com/weilin1205/CorporateEvasionDecoder

1 Research Question and Motivation

Corporate earnings calls are a primary channel through which public companies communicate financial performance to investors and analysts. During the Q&A portion, executives may respond with varying degrees of transparency: some answers are direct and data-driven, while others employ hedging language, excessive jargon, or strategic vagueness to deflect difficult questions. Detecting such evasive communication is valuable for financial analysts, regulators, and NLP researchers.

I pose the following research question: *Can I automatically classify corporate Q&A responses as* DIRECT, EVASIVE, *or* JARGON*-heavy using a combination of handcrafted linguistic features and text representations?*

This task is challenging because evasiveness is subjective, corporate jargon is domain-specific, and the dataset is modest (n = 1,129), making high-capacity models prone to overfitting—an ideal testbed for comparing classical ML against a fine-tuned transformer.

2 Dataset Documentation

2.1 Data Type and Source

Each sample is a **question–answer pair** extracted from public earnings call transcripts. The text is in English and drawn from formal corporate communication. Transcripts were obtained from **SeekingAlpha** via the RapidAPI endpoint `seeking-alpha.p.rapidapi.com` [5].

2.2 Amount and Composition

The dataset spans **20 companies** across **6 sectors** (Table 1). A total of 131 transcripts were crawled, of which 87 contained extractable Q&A sections, yielding 1,129 pairs with a well-balanced class distribution (≈33% each).

2.3 Collection Process

Data collection followed a four-stage pipeline:

Table 1: Dataset composition by label and sector.

(a) Class distribution.

Label	Count	Prop.
DIRECT	395	35.0%
EVASIVE	352	31.2%
JARGON	382	33.8%
Total	1,129	100.0%

(b) Sector distribution.

Sector	Tickers	#
Technology	AAPL, MSFT, GOOGL, META, AMZN, NVDA	6
Finance	JPM, GS, BAC	3
Healthcare	JNJ, PFE, UNH	3
Consumer	PG, KO, WMT, MCD	4
Energy	XOM, CVX	2
Industrial	CAT, GE	2
Total		**20**

1. **Crawling.** For each of the 20 tickers, up to 8 recent earnings call transcript IDs were retrieved via the SeekingAlpha API. Full HTML transcripts were then downloaded (131 total, capped at 480 API calls).
2. **Q&A extraction.** HTML transcripts were parsed to isolate the Q&A section. Speaker turns were segmented by `<p>` tags with speaker-name formatting. Analyst questions were paired with the subsequent executive answer, yielding structured `(question, answer, ticker, date)` tuples.
3. **LLM annotation.** Each Q&A pair was classified by **Qwen3.5-9B** [6] running locally on an NVIDIA RTX A6000 GPU with zero-shot prompting. The prompt asked the model to assign exactly one of three labels—DIRECT, EVASIVE, or JARGON—based on explicit criteria (e.g., hedging language, numerical specificity, buzzword density). This semi-automatic approach enabled labeling the full dataset consistently while acknowledging that label noise is inevitable.
4. **Feature engineering.** Two feature families were computed: 500-dimensional TF-IDF vectors (unigrams + bigrams) and 21 handcrafted linguistic features (see Section 3).

2.4 Conditions and Limitations

Transcripts are public, sourced from a commercial API, and contain no personally identifiable information beyond public executive names. The LLM-generated labels introduce systematic noise: the annotator has no ground-truth human labels for calibration. I acknowledge this as the primary limitation and discuss its impact on model performance in Section 6.

2.5 Examples

- **Direct:** *"Revenue grew 12% year-over-year to $23.4 billion, driven primarily by cloud services which grew 28%."*
- **Evasive:** *"I continue to evaluate our strategic options and remain focused on long-term value creation for our stakeholders across the portfolio."*
- **Jargon:** *"We're leveraging our synergistic ecosystem to drive paradigm-shifting innovation across verticals."*

3 Feature Engineering

The feature space comprises **521 dimensions**: 500 TF-IDF features and 21 handcrafted linguistic features designed to capture communication style beyond bag-of-words statistics.

Table 2: The 21 handcrafted linguistic features grouped by category.

Category	Feature	Description
Length	`answer_word_count`	Word count of the answer
	`answer_sent_count`	Sentence count of the answer
	`avg_sent_length`	Average sentence length (words)
	`question_word_count`	Word count of the question
	`qa_length_ratio`	Answer length / question length
Hedging	`hedge_word_count`	Count of hedge words (*might, could, possibly, generally*, etc.)
	`hedge_word_ratio`	Proportion of hedge words
Jargon	`jargon_word_count`	Count of corporate jargon (*synergy, leverage, ecosystem*, etc.)
	`jargon_word_ratio`	Proportion of jargon words
Specificity	`number_count`	Count of numeric tokens
	`number_ratio`	Proportion of numeric tokens
	`metric_count`	Count of metric phrases (e.g., *$X billion*)
	`metric_ratio`	Proportion of metric phrases
Pronouns	`first_person_ratio`	Ratio of first-person pronouns
	`third_person_ratio`	Ratio of third-person pronouns
Complexity	`avg_word_length`	Average word length (characters)
	`long_word_ratio`	Proportion of words ≥ 7 characters
Structure	`starts_with_direct_answer`	Binary: starts with yes/no/number
	`question_keyword_overlap`	Jaccard overlap of Q&A keywords
Semantic	`qa_tfidf_similarity`	Cosine similarity between Q&A TF-IDF vectors

The TF-IDF [7] features were extracted from the concatenated question–answer text using scikit-learn's `TfidfVectorizer` [1] with a maximum of 500 features (unigrams and bigrams). The handcrafted features capture stylistic and structural properties that are hypothesized to discriminate between direct, evasive, and jargon-heavy responses.

4 Methods

I evaluate four classifiers spanning a spectrum from linear models to deep pre-trained transformers. All classical models use the full 521-feature representation; DistilBERT operates on raw text.

Logistic Regression (LR). Multinomial LR with L2 regularization ($C = 1.0$, L-BFGS solver). Serves as a transparent linear baseline with interpretable coefficients.

Support Vector Machine (SVM). RBF-kernel SVM ($C = 1.0$, $\gamma = \text{scale}$) with one-vs-rest decomposition and Platt scaling for probability estimates.

XGBoost. Gradient-boosted trees [2]: 100 estimators, depth 4, learning rate 0.1. Built-in gain-based feature importance is used for ablation analysis.

DistilBERT. Fine-tuned `distilbert-base-uncased` [3] via Hugging Face Transformers [8] with a linear head, class-weighted cross-entropy loss, learning rate 5×10^{-5}, dropout 0.3, and early stopping (patience 3, up to 10 epochs).

Evaluation Protocol. All models were evaluated using **stratified 5-fold cross-validation** to ensure each fold preserves the class distribution. I report accuracy, macro-averaged F1 score, and one-vs-rest AUROC. Per-class precision, recall, and F1 are reported for detailed analysis.

5 Experiments and Results

5.1 Experiment 1: Baseline Comparison

Table 3 presents the baseline performance of all four models on the full 521-feature set (or raw text for DistilBERT).

Table 3: Baseline 5-fold cross-validation results (mean ± std).

Model	Accuracy	F1 (macro)	AUROC
Logistic Regression	0.455 ± 0.027	0.453 ± 0.026	0.646
SVM (RBF)	0.532 ± 0.037	0.528 ± 0.036	0.731
XGBoost	0.544 ± 0.029	0.542 ± 0.030	0.728
DistilBERT	$\mathbf{0.582 \pm 0.031}$	$\mathbf{0.580 \pm 0.031}$	$\mathbf{0.745 \pm 0.031}$

Table 4: Per-class precision, recall, and F1 for all classical models (5-fold CV averages).

Model	Class	Precision	Recall	F1
Logistic Regression	Direct	0.520	0.524	0.522
	Evasive	0.411	0.423	0.417
	Jargon	0.427	0.411	0.419
SVM (RBF)	Direct	0.580	0.595	0.588
	Evasive	0.514	0.480	0.496
	Jargon	0.496	0.513	0.505
XGBoost	Direct	0.571	0.603	0.586
	Evasive	0.541	0.509	0.524
	Jargon	0.517	0.516	0.516

DistilBERT achieves the highest performance, demonstrating that pre-trained contextual representations provide an advantage over handcrafted features for this task. Among classical models, XGBoost leads marginally over SVM, while Logistic Regression lags substantially.

Table 4 breaks down per-class performance across all classical models. DIRECT is consistently the easiest class (F1 $\approx$ 0.59), while EVASIVE and JARGON overlap significantly (F1 $\approx$ 0.50–0.52).

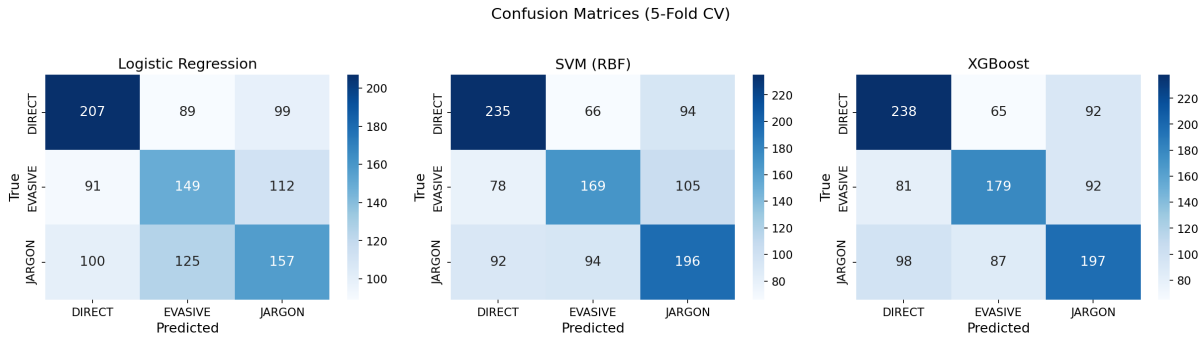

Figure 1: Confusion matrices for classical models.

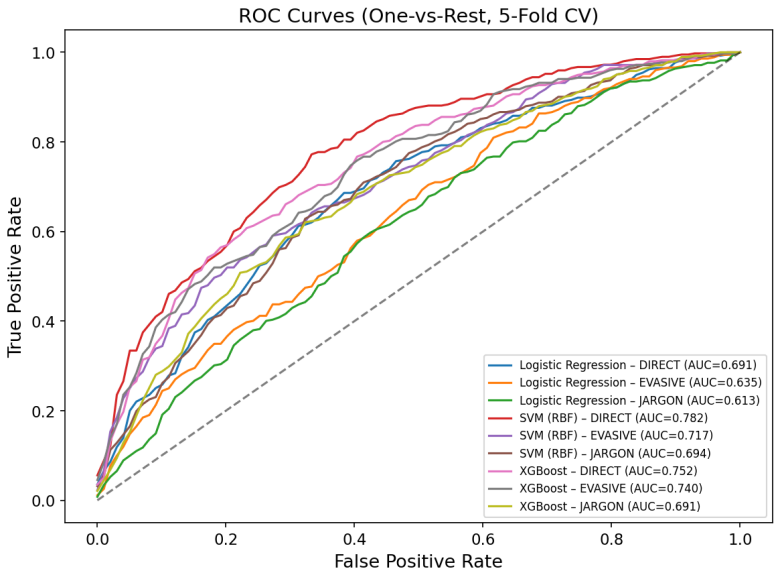

Figure 2: One-vs-rest ROC curves.

The confusion matrices (Figure 1) reveal that EVASIVE and JARGON are frequently confused with each other, consistent with our intuition that both involve non-specific, non-committal language. The ROC curves (Figure 2) show that all models achieve AUROC well above 0.5 for each class, confirming that the features carry discriminative signal despite the modest overall accuracy.

5.2 Experiment 2: Learning Curves

Figure 3 shows that all models exhibit a persistent training–validation gap, indicating overfitting. Validation F1 improves as more data is added but plateaus around 0.55,

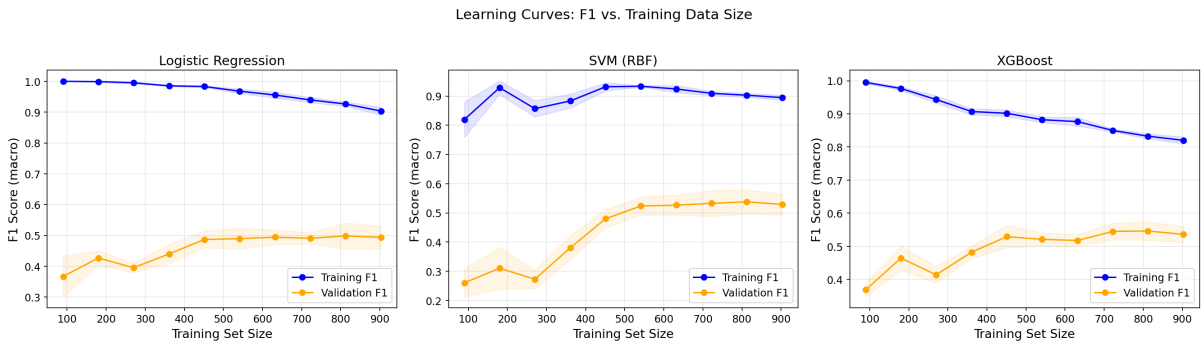

Figure 3: Learning curves showing training vs. validation F1 as a function of training set fraction.

suggesting that performance is bounded not only by model capacity but also by label noise from the LLM annotator. The upward trend at larger training fractions indicates that *more data would likely help*; this hypothesis is confirmed by the LLM data augmentation experiment (Section 5.6), which yields 10–12 F1-point gains.

5.3 Experiment 3: SMOTE Analysis

Since class imbalance can degrade minority-class performance, I evaluated SMOTE [4] (Synthetic Minority Oversampling Technique) applied within each cross-validation fold.

Table 5: Effect of SMOTE on macro-F1.

Model	No SMOTE	With SMOTE	Δ
Logistic Regression	0.453	0.453	−0.000
SVM (RBF)	0.528	0.523	−0.005
XGBoost	0.542	0.546	+0.004

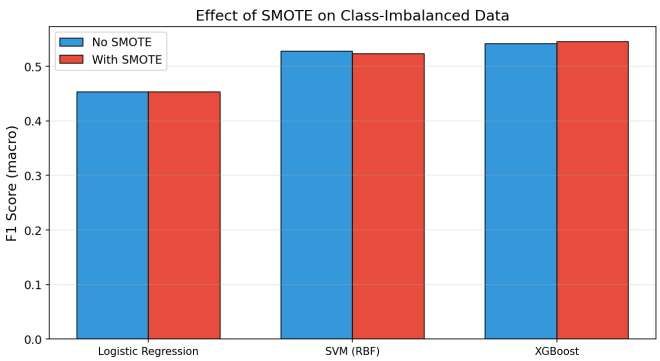

Figure 4: SMOTE vs. no-SMOTE comparison across models.

SMOTE produces negligible changes ($|\Delta| < 0.006$) for all models. This is expected: the dataset is already well-balanced (35/31/34%), so oversampling the slightly smaller EVASIVE class adds little value. For SVM, the synthetic samples even introduce a slight performance drop, possibly because SMOTE-generated points near the decision boundary add noise in high-dimensional feature space. This experiment confirms that class imbalance is *not* a contributing factor to the moderate accuracy.

5.4 Experiment 4: PCA Dimensionality Reduction

With 521 features and only 1,129 samples, the curse of dimensionality may harm linear models. I applied PCA to the feature matrix and evaluated performance at various component counts.

Table 6: Macro-F1 as a function of PCA components.

Components	LR	SVM	XGBoost
25	0.538	0.528	0.513
50	**0.545**	**0.534**	0.529
100	0.519	0.527	0.516
200	0.497	0.524	0.509
521 (full)	0.453	0.528	**0.542**

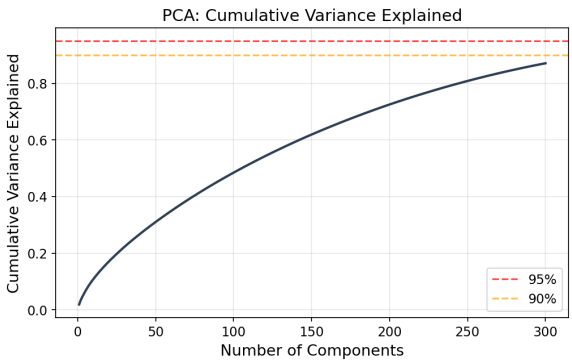

Figure 5: Cumulative explained variance by PCA.

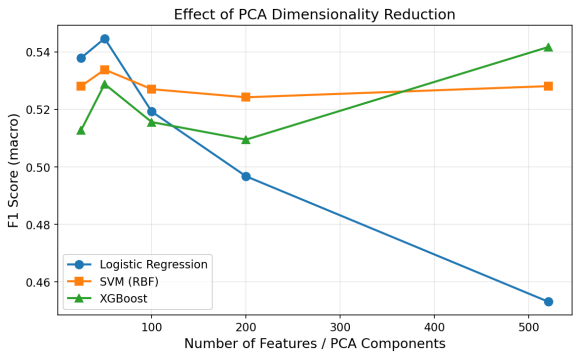

Figure 6: Model performance vs. PCA components.

The key finding is that **PCA-50 dramatically improves Logistic Regression** from F1 = 0.453 to 0.545 (+20%), making it competitive with SVM and XGBoost. This reveals that LR's poor baseline performance stems not from insufficient model capacity but from overfitting to high-dimensional noise. PCA acts as a regularizer by projecting out low-variance (likely noisy) directions.

SVM is relatively robust across dimensionalities, consistent with the RBF kernel's implicit regularization through $\gamma = \text{scale}$. XGBoost performs best with all 521 features because its tree-based structure performs built-in feature selection by only splitting on informative features.

5.5 Experiment 5: Feature Ablation

To quantify the contribution of each feature family, I compared TF-IDF–only, handcrafted–only, and combined feature sets.

The most striking result is that **21 handcrafted features outperform 500 TF-IDF features** for Logistic Regression (0.518 vs. 0.439) and match SVM performance (0.511 vs. 0.510). This underscores the value of domain-informed feature engineering: the handcrafted features distill the linguistically relevant signal into a compact, low-noise representation that linear models can exploit effectively.

Table 7: Feature ablation: macro-F1 by feature subset.

Feature Set	LR	SVM	XGBoost
TF-IDF Only (500)	0.439	0.510	0.508
Handcrafted Only (21)	**0.518**	0.511	0.513
Combined (521)	0.453	0.528	**0.542**

For XGBoost, combining both feature families yields the best result (0.542), suggesting that the ensemble learner benefits from both the fine-grained lexical signal in TF-IDF and the high-level stylistic signal in handcrafted features. The tree-based model can selectively attend to informative features from either family while ignoring noise.

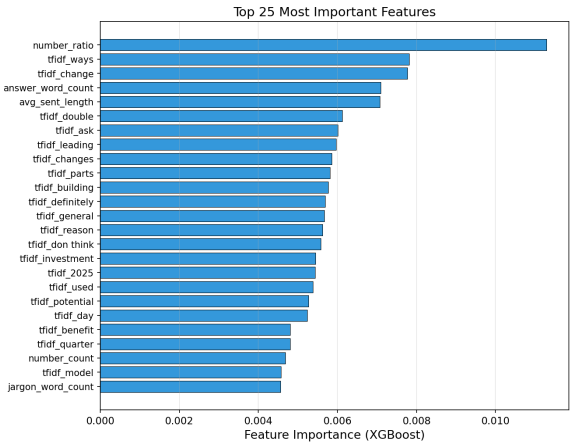

Figure 7: Top features by XGBoost gain (full set).

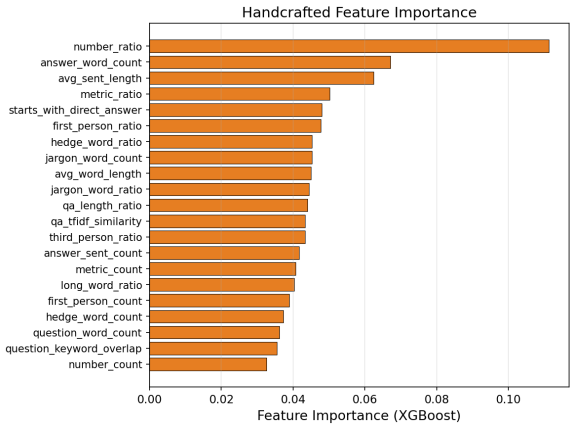

Figure 8: Handcrafted feature importance.

Feature importance analysis (Figures 7 and 8) reveals that hedging and jargon ratios, question–answer length ratio, and semantic similarity are among the most discriminative handcrafted features—consistent with our intuitions about what characterizes evasive communication.

5.6 Experiment 6: LLM-Based Data Augmentation

Motivated by the learning curve analysis showing that more data improves performance, I explored LLM-based data augmentation using Qwen3.5-9B. For each training sample, the model was prompted to rewrite answers with a different communication style (e.g., Evasive → Direct and vice versa), generating new training samples with known labels. This approach produces semantically meaningful augmentations rather than surface-level perturbations.

To isolate the effect of augmented text on lexical representations, this experiment uses TF-IDF-only features (500 dimensions) rather than the full 521-feature set, so that the newly generated answers contribute directly to the vocabulary-level feature space. Table 8 and Figure 9 summarize the results.

The improvements are striking: all three models gain 10–12 F1 points, with LR benefiting most (+12.4%). These gains far exceed those from SMOTE or PCA, indicating that the bottleneck was not feature engineering or imbalance, but rather the quantity and diversity of labeled training text. The LLM-generated rewrites expose the classifiers to a

Table 8: Effect of LLM data augmentation on macro-F1 (TF-IDF features).

Model	Original	+ Augmented	Δ
Logistic Regression	0.439	0.563	**+0.124**
SVM (RBF)	0.510	0.612	**+0.102**
XGBoost	0.508	0.626	**+0.117**

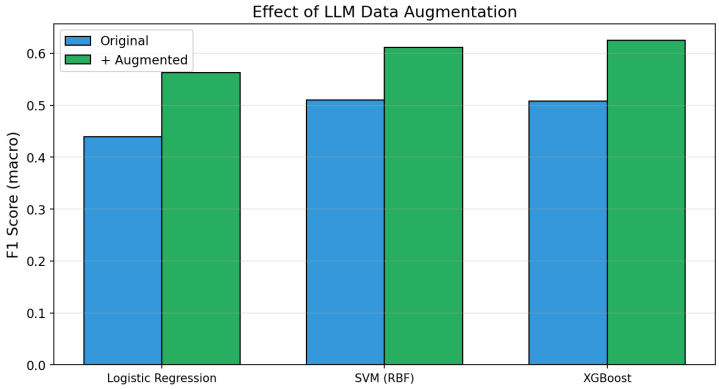

Figure 9: F1 score comparison with and without LLM data augmentation.

broader range of syntactic phrasings for each class label, reducing overfitting to specific lexical patterns in the original 1,129 samples.

6 Discussion

6.1 Expected vs. Observed Results

Contrary to expectation, 21 handcrafted features outperformed 500 TF-IDF features for Logistic Regression and matched SVM, illustrating that *fewer, higher-quality features often outperform many noisy ones* when data is limited. DistilBERT's advantage over the best classical model (F1 = 0.580 vs. 0.542) was expected but modest, as LLM-generated label noise constrains all methods equally. The most surprising result was the magnitude of the augmentation gain (+10–12 F1 points), confirming that data quantity—not model capacity—is the primary bottleneck.

6.2 Factors Affecting Performance

Label noise. LLM-generated zero-shot labels inevitably contain systematic errors for ambiguous cases where evasiveness and jargon overlap. Human annotation would improve absolute performance but was infeasible at scale.

Class overlap. Evasive and Jargon both involve non-specific language, differing only in mechanism (hedging vs. buzzwords), which explains their frequent confusion in all models.

Dataset size. At n = 1,129, overfitting is a concern for high-dimensional TF-IDF; PCA and augmentation both address this from different angles (dimensionality reduction

vs. data expansion).

6.3 Key Takeaways

1. **Domain features beat TF-IDF for linear models.** 21 handcrafted features outperformed 500 TF-IDF features for LR, demonstrating the value of injecting domain knowledge.
2. **PCA as regularization.** PCA-50 improved LR by 20%, revealing that the baseline's weakness was overfitting, not insufficient model capacity.
3. **Diagnose before fixing.** SMOTE had negligible impact (data already balanced); the real bottleneck was data quantity, confirmed by the 10–12 F1-point gains from augmentation.
4. **Data is the dominant constraint.** LLM augmentation yielded the largest gains of any experiment—larger than architecture choice, feature engineering, or resampling.

6.4 Future Work

Key next steps include: (1) human annotation of a subset to calibrate the LLM labeler; (2) domain-specific models such as FinBERT with broader hyperparameter search; (3) expanding coverage to more companies and fiscal years; and (4) semi-supervised learning over the large pool of unlabeled transcript text.

References

[1] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD*, pp. 785–794, 2016.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[5] SeekingAlpha, "Earnings call transcripts," https://seekingalpha.com, accessed 2026.

[6] Qwen Team, "Qwen3 Technical Report," Alibaba Cloud, *arXiv preprint arXiv:2505.09388*, 2025.

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[8] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP 2020: System Demonstrations*, pp. 38–45, 2020.

Appendix: Program Code

The pipeline consists of six scripts executed in order:

- 01_crawl_transcripts.py
- 02_extract_qa_pairs.py
- 03_llm_annotate.py
- 04_build_dataset.py
- 05_experiments.py
- 06_augment_data.py

Selected key components from each script are shown below. Full source is available at https://github.com/weilin1205/CorporateEvasionDecoder.

Script 1: `01_crawl_transcripts.py` — Transcript Crawling

This script fetches earnings call transcripts from SeekingAlpha via RapidAPI, with retry logic and an API call budget cap.

```python
MAX_API_CALLS = 480    # hard cap to avoid exceeding quota

def api_get(path: str, retries: int = 3) -> dict:
    """Send a GET request to the SeekingAlpha RapidAPI endpoint.
    Retries up to `retries` times with exponential back-off on failure.
    Raises RuntimeError if the global API call budget is exhausted."""
    global api_call_count
    if api_call_count >= MAX_API_CALLS:
        raise RuntimeError(f"API call limit reached ({MAX_API_CALLS})")
    for attempt in range(retries):
        try:
            conn = http.client.HTTPSConnection(RAPIDAPI_HOST)
            conn.request("GET", path, headers=HEADERS)
            res = conn.getresponse()
            raw = res.read()
            conn.close()
            api_call_count += 1
            decoded = raw.decode("utf-8").strip()
            if not decoded:
                raise ValueError(f"Empty response (HTTP {res.status})")
            return json.loads(decoded)
        except (json.JSONDecodeError, ValueError) as exc:
            wait = 2 ** (attempt + 1)       # 2s, 4s, 8s
            time.sleep(wait)
    return {}
```

Listing 1: API request helper with retry and call-count enforcement.

```python
def crawl_all():
    df = pd.read_csv(TICKERS_PATH)
    for _, row in df.iterrows():
        ticker = row["Ticker"]
        items = get_transcript_list(ticker)

        # Keep only earnings-related transcripts by title keywords
        earnings = [
            x for x in items
            if x.get("type") == "transcript"
            and any(kw in (x.get("attributes") or {}).get("title", "").lower()
                    for kw in ["earnings call", "earnings conference",
                               "quarterly results", "financial results"])
        ]

        for item in earnings[:TRANSCRIPTS_PER_TICKER]:
            details  = get_transcript_details(str(item["id"]))
            html     = details["data"]["attributes"].get("content", "")
            # Convert HTML to plain text for downstream processing
            content_text = extract_text_from_html(html) if html else ""
            record = {
                "id": str(item["id"]), "ticker": ticker,
                "content_text": content_text, "content_html": html,
            }
            # Persist raw record; later scripts read from this directory
            with open(out_path, "w", encoding="utf-8") as f:
                json.dump(record, f, indent=2, ensure_ascii=False)
```

Listing 2: Main crawl loop: filter earnings calls and save raw JSON.

Script 2: `02_extract_qa_pairs.py` — Q&A Pair Extraction

Speaker turns are classified as *analyst* or *executive*; consecutive analyst turns are paired with the following executive response.

```python
def pair_qa_segments(segments, exec_names, analyst_names, record) -> list[dict]:
    """Scan speaker-turn segments and emit (question, answer) pairs.
    Skips operator turns; merges consecutive same-analyst turns into
    one question block; collects all subsequent executive turns as the answer."""
    pairs = []
    i = 0
    while i < len(segments):
        seg = segments[i]
        role = classify_speaker(seg["speaker"], exec_names, analyst_names)

        if role == "operator":            # skip queue/transition messages
```

```
            i += 1; continue

        if role == "analyst":
            question_parts = [seg["text"]]
            j = i + 1
            # Merge follow-up turns from the same analyst
            while j < len(segments):
                r = classify_speaker(segments[j]["speaker"],
                                     exec_names, analyst_names)
                if r == "analyst" and segments[j]["speaker"] == seg["speaker"]:
                    question_parts.append(segments[j]["text"]); j += 1
                else:
                    break

            # Collect the executive answer (may span multiple turns)
            answer_parts = []
            while j < len(segments):
                r = classify_speaker(segments[j]["speaker"],
                                     exec_names, analyst_names)
                if r in ("executive", "unknown"):
                    answer_parts.append(segments[j]["text"]); j += 1
                elif r in ("operator", "analyst"):
                    break
                else:
                    j += 1

            q = " ".join(question_parts)
            a = " ".join(answer_parts)
            # Quality filter: require minimum token lengths
            if len(q.split()) >= 8 and len(a.split()) >= 15:
                pairs.append({"question": q, "answer": a,
                              "ticker": record.get("ticker", ""),
                              "publish_date": record.get("publishOn", "")})
            i = j
        else:
            i += 1
    return pairs
```

Listing 3: Pairing analyst questions with executive answers.

Script 3: `03_llm_annotate.py` — LLM Annotation

Each Q&A pair is classified by a locally-run Qwen3.5-9B using a zero-shot system prompt with explicit label criteria.

```
# System prompt defines labeling criteria unambiguously
SYSTEM_PROMPT = (
    "Classify the executive's earnings-call response as exactly one of: "
    "DIRECT, EVASIVE, or JARGON. Output ONLY that single word, nothing else.\n\n"
    "DIRECT  = specific numbers, facts, or clear explanations that address "
    "the question.\n"
```

```python
    "EVASIVE = deflects, redirects, gives a non-answer, or avoids the "
    "question.\n"
    "JARGON  = excessive buzzwords or vague corporate language hiding lack "
    "of substance."
)

def parse_label(raw: str) -> int:
    """Extract integer label (0/1/2) from raw model output.
    Handles Qwen3 thinking-block artifacts and partial matches."""
    text = strip_thinking(raw).upper().strip()  # remove <think>...</think>
    first_word = re.sub(r"[^A-Z]", "", text.split()[0]) if text.split() else ""
    if first_word in LABEL_MAP:           # clean single-word output
        return LABEL_MAP[first_word]
    for label_name, label_id in LABEL_MAP.items():  # substring fallback
        if label_name in text:
            return label_id
    return -1   # unparseable; row will be dropped
```

Listing 4: Zero-shot annotation prompt and label extraction.

Script 4: `04_build_dataset.py` — Feature Engineering

The 21 handcrafted features are computed per Q&A pair; TF-IDF is fit on all answer texts. The two feature families are then concatenated.

```python
def extract_handcrafted_features(question: str, answer: str) -> dict:
    """Return a dict of 21 scalar features capturing length, hedging,
    jargon density, numeric specificity, pronoun usage, and Q-A similarity."""
    q_words = question.lower().split()
    a_words  = answer.lower().split()
    a_sents  = [s for s in re.split(r"[.!?]+", answer) if s.strip()]
    a_len = max(len(a_words), 1)

    # --- Hedging: words that soften commitment (might, could, possibly, ...) ---
    hedge_count = sum(1 for w in a_words if w in HEDGE_WORDS)

    # --- Jargon: corporate buzzwords (synergy, leverage, ecosystem, ...) ------
    jargon_count = sum(1 for w in a_words if w in JARGON_WORDS)

    # --- Specificity: numeric tokens and domain metric phrases ----------------
    number_count = sum(1 for w in a_words if re.match(r"[\d$%,.]+$", w))
    metric_count = sum(1 for w in a_words if w in CONCRETE_METRICS)

    # --- Structure: does the answer open with a direct yes/no/number? ---------
    first_10 = " ".join(a_words[:10])
```

```python
    starts_direct = int(any(first_10.startswith(s) for s in DIRECT_STARTERS))

    # --- Semantic overlap: Jaccard similarity of content words ----------------
    q_cw = {w for w in q_words if len(w) > 3}
    a_cw = {w for w in a_words if len(w) > 3}
    keyword_overlap = len(q_cw & a_cw) / max(len(q_cw), 1)

    return {
        "answer_word_count":   a_len,
        "avg_sent_length":     a_len / max(len(a_sents), 1),
        "qa_length_ratio":     a_len / max(len(q_words), 1),
        "hedge_word_ratio":    hedge_count / a_len,
        "jargon_word_ratio":   jargon_count / a_len,
        "number_ratio":        number_count / a_len,
        "metric_ratio":        metric_count / a_len,
        "first_person_ratio":  sum(1 for w in a_words
                                   if w in {"i","we","our","us","my"}) / a_len,
        "avg_word_length":     np.mean([len(w) for w in a_words]),
        "starts_with_direct_answer": starts_direct,
        "question_keyword_overlap":  keyword_overlap,
        # ... (10 additional ratio/count features follow the same pattern)
    }
```

Listing 5: Handcrafted linguistic feature extraction (key dimensions).

```python
# TF-IDF on answer text: 500 features, unigrams + bigrams
tfidf = TfidfVectorizer(max_features=500, stop_words="english",
                        ngram_range=(1, 2), min_df=2, max_df=0.95)
X_tfidf = tfidf.fit_transform(df["answer"].fillna(""))  # sparse (n, 500)

# Q-A cosine similarity as a single additional feature
tfidf_qa = TfidfVectorizer(max_features=5000, stop_words="english")
tfidf_qa.fit(df["question"].tolist() + df["answer"].tolist())
Q_vecs = tfidf_qa.transform(df["question"].fillna(""))
A_vecs = tfidf_qa.transform(df["answer"].fillna(""))
qa_sim = np.array([cosine_similarity(Q_vecs[i], A_vecs[i])[0, 0]
                   for i in range(len(df))]).reshape(-1, 1)

# Stack: [TF-IDF (500) | handcrafted (20) | qa_similarity (1)] = 521 dims
X_combined = sparse.hstack([X_tfidf,
                            sparse.csr_matrix(
                                np.hstack([X_handcrafted, qa_sim]))
                                ])
```

Listing 6: Combining TF-IDF and handcrafted features into one sparse matrix.

Script 5: `05_experiments.py` — ML Experiments

All classical models share a common `StandardScaler + Classifier` pipeline evaluated with stratified 5-fold cross-validation.

```python
def get_models():
    """Return the three classical classifiers with fixed hyperparameters."""
    return {
        "Logistic Regression": LogisticRegression(
            C=1.0, max_iter=2000, solver="lbfgs",
            multi_class="multinomial", random_state=42),
        "SVM (RBF)": SVC(
            C=1.0, kernel="rbf", probability=True, random_state=42),
        "XGBoost": XGBClassifier(
            n_estimators=100, max_depth=4, learning_rate=0.1,
            eval_metric="mlogloss", random_state=42, n_jobs=-1),
    }

def exp_baseline(X, y):
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scoring = {"accuracy": "accuracy",
               "f1_macro": make_scorer(f1_score, average="macro")}

    for name, model in get_models().items():
        # Pipeline ensures the scaler is re-fit inside each fold
        pipe = Pipeline([("scaler", StandardScaler(with_mean=False)),
                         ("clf", model)])
        cv = cross_validate(pipe, X, y, cv=skf,
                            scoring=scoring, return_train_score=True)
        print(f"{name}: Acc={cv['test_accuracy'].mean():.4f} "
              f"F1={cv['test_f1_macro'].mean():.4f}")
```

Listing 7: Baseline cross-validation loop for classical models.

```python
def exp_pca(X, y):
    """Evaluate each model at multiple PCA component counts (25/50/100/200/full)
    to diagnose overfitting in high-dimensional feature space."""
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    for nc in [25, 50, 100, 200]:
        for name, model in get_models().items():
            # PCA is placed inside the pipeline so it never sees test-fold data
            pipe = Pipeline([
                ("scaler", StandardScaler(with_mean=False)),
                ("pca",    PCA(n_components=nc, random_state=42)),
                ("clf",    model),
            ])
            s = cross_validate(pipe, X, y, cv=skf,
                               scoring={"f1": make_scorer(f1_score,
                                                          average="macro")})
            print(f"PCA-{nc} | {name}: F1={s['test_f1'].mean():.4f}")
```

Listing 8: PCA dimensionality reduction experiment across component counts.

```python
# Key training settings
NUM_EPOCHS, LR, BATCH_SIZE = 10, 5e-5, 16
loss_fn = torch.nn.CrossEntropyLoss(weight=class_weights.to(device))

for epoch in range(NUM_EPOCHS):
```

```python
    model.train()
    for batch_inputs, batch_labels in train_loader:
        outputs = model(**{k: v.to(device) for k, v in batch_inputs.items()})
        loss    = loss_fn(outputs.logits, batch_labels.to(device))
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step(); scheduler.step(); optimizer.zero_grad()

    # Early stopping: track best validation F1 across epochs
    val_f1 = evaluate_f1(model, test_loader, device)
    if val_f1 > best_val_f1:
        best_val_f1 = val_f1
        best_state  = {k: v.cpu().clone()
                       for k, v in model.state_dict().items()}
        patience_counter = 0
    else:
        patience_counter += 1
        if patience_counter >= 3:    # patience = 3
            break                    # restore best checkpoint after loop
```

Listing 9: DistilBERT fine-tuning with early stopping (simplified).

Script 6: `06_augment_data.py` — LLM Data Augmentation

New training samples are generated by prompting Qwen3.5-9B to rewrite EVASIVE answers as DIRECT and vice versa.

```python
# Prompt template for EVASIVE     DIRECT rewriting
REWRITE_EVASIVE_TO_DIRECT = """\
You are an expert corporate communication editor. Below is an analyst \
question and an evasive executive response. Rewrite ONLY the executive's \
response to be DIRECT: address the question head-on with specific, concrete \
information. Keep the same topic and approximate length.
Output ONLY the rewritten response, nothing else.

Question: {question}
Evasive Response: {answer}
Direct Rewrite:"""

def generate_rewrite(prompt_text: str) -> str:
    """Run the LLM and return the rewritten answer, stripping any
    Qwen3 chain-of-thought <think> blocks from the output."""
    inputs = tokenizer(prompt_text, return_tensors="pt",
                       truncation=True, max_length=4096).to(model.device)
    with torch.no_grad():
        output_ids = model.generate(**inputs, max_new_tokens=512,
                                    temperature=0.5, do_sample=True)
    new_tokens = output_ids[0][inputs["input_ids"].shape[1]:]
    raw = tokenizer.decode(new_tokens, skip_special_tokens=True)
    return strip_thinking(raw)    # removes <redacted_thinking>...</> blocks
```

```python

def is_valid_rewrite(text: str) -> bool:
    """Reject outputs that leaked prompt text, reasoning scaffolding,
    or are too short to be a coherent executive response."""
    if len(text.split()) < 10: return False
    low = text.lower()
    if "you are an expert corporate communication editor" in low:
        return False
    if "redacted_thinking" in low: return False
    return True
```

Listing 10: Rewrite prompts and generation call for data augmentation.

Configuration: `config.py`

```python
# Centralized configuration for reproducibility
RAPIDAPI_HOST = "seeking-alpha.p.rapidapi.com"
MAX_API_CALLS = 480
TRANSCRIPTS_PER_TICKER = 8

LLM_MODEL_PATH = os.environ.get("Qwen/Qwen3.5-9B")
LLM_BATCH_SIZE = 1  # single-sample inference for reliability
LLM_MAX_NEW_TOKENS = 256  # room for thinking tags + label

CV_FOLDS = 5
RANDOM_STATE = 42
TFIDF_MAX_FEATURES = 500
PCA_COMPONENTS_LIST = [25, 50, 100, 200]
LEARNING_CURVE_FRACTIONS = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
```

Listing 11: Core configuration knobs.